Unit V: Project Management

Software Project Management- Software Configuration Management – Project Scheduling- DevOps: Motivation-Cloud as a platform-Operations- Deployment Pipeline: Overall Architecture Building and Testing-Deployment- Tools- Case Study

Software Project Management

The main goal of software project management is to enable a group of developers to work effectively towards the successful completion of a project.

1.RESPONSIBILITIES OF A SOFTWARE PROJECT MANAGER

We can broadly classify a project manager's varied responsibilities into the following two major categories:

1. Project planning, and
2. Project monitoring and control.

1.1 Job Responsibilities for Managing Software Projects

Project planning: Project planning is undertaken immediately after the feasibility study phase and before the starting of the requirements analysis and specification phase.

The initial project plans are revised from time to time as the project progresses and more project data become available.

Project monitoring and control: Project monitoring and control activities are undertaken once the development activities start.

- As the project gets underway, the details of the project that were unclear earlier at the start of the project emerge and situations that were not visualised earlier arise.
- While carrying out project monitoring and control activities, a project manager usually needs to change the plan to cope up with specific situations at hand.

1.2 Skills Necessary for Managing Software Projects

Three skills that are most critical to successful project management are the following:

1. Knowledge of project management techniques.
2. Decision taking capabilities.
3. Previous experience in managing similar projects.

2.PROJECT PLANNING

During project planning, the project manager performs the following activities.

1.Estimation: The following project attributes are estimated.

Cost: How much is it going to cost to develop the software product?

Duration: How long is it going to take to develop the product?

Effort: How much effort would be necessary to develop the product?

- The effectiveness of all later planning activities such as scheduling and staffing are

dependent on the accuracy with which these three estimations have been made.

2.Scheduling: After all the necessary project parameters have been estimated, the schedules for manpower and other resources are developed.

3.Staffing: Staff organisation and staffing plans are made.

4.Risk management: This includes risk identification, analysis, and abatement planning.

5.Miscellaneous plans: This includes making several other plans such as quality assurance plan, and configuration management plan, etc.

Figure 3.1 shows the order in which the planning activities are undertaken. Observe that size estimation is the first activity that a project manager undertakes during project planning.

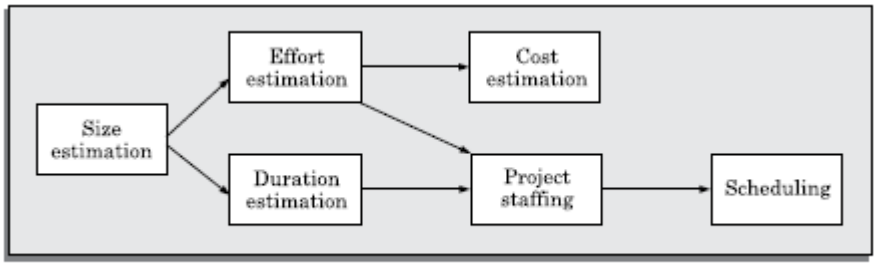


FIGURE 3.1 Precedence ordering among planning activities.

- Size is the most fundamental parameter, based on which all other estimations and project

plans are made.

- As can be seen from Figure 3.1, based on the size estimation, the effort required to complete a project and the duration over which the development is to be carried out are estimated.
- Based on the effort estimation, the cost of the project is computed. The estimated cost forms the basis on which price negotiations with the customer is carried out.
- Other planning activities such as staffing, scheduling etc. are undertaken based on the effort and duration estimates made.

2.1 Sliding Window Planning

- In the sliding window planning technique, starting with an initial plan, the project is planned more accurately over a number of stages.
- It is usually very difficult to make accurate plans for large projects at project initiation.
- A part of the difficulty arises from the fact that large projects may take several years to complete.
- As a result, during the span of the project, the project parameters, scope of the project, project staff, etc., often change drastically resulting in the initial plans going haywire.
- In order to overcome this problem, sometimes project managers undertake project planning over several stages.
- That is, after the initial project plans have been made, these are revised at frequent intervals.
- Planning a project over a number of stages protects managers from making big commitments at the start of the project. This technique of staggered planning is known as sliding window planning.

2.2 The SPMP Document of Project Planning

Once project planning is complete, project managers document their plans in a software project management plan (SPMP) document. Listed below are the different items that the SPMP document should discuss. This list can be used as a possible organisation of the SPMP document.

Organisation of the software project management plan (SPMP) document

1. Introduction
   (a) Objectives
   (b) Major Functions
   (c) Performance Issues
   (d) Management and Technical Constraints
2. Project estimates
   (a) Historical Data Used
   (b) Estimation Techniques Used
   (c) Effort, Resource, Cost, and Project Duration Estimates
3. Schedule
   (a) Work Breakdown Structure

(b) Task Network Representation
(c) Gantt Chart Representation
(d) PERT Chart Representation

4. Project resources
    (a) People
    (b) Hardware and Software
    (c) Special Resources
5. Staff organisation
    (a) Team Structure
    (b) Management Reporting
6. Risk management plan
    (a) Risk Analysis
    (b) Risk Identification
    (c) Risk Estimation
    (d) Risk Abatement Procedures
7. Project tracking and control plan
    (a) Metrics to be tracked
    (b) Tracking plan
    (c) Control plan
8. Miscellaneous plans
    (a) Process Tailoring
    (b) Quality Assurance Plan
    (c) Configuration Management Plan
    (d) Validation and Verification
    (e) System Testing Plan
    (f) Delivery, Installation, and Maintenance Plan

3. METRICS FOR PROJECT SIZE ESTIMATION

The project size is a measure of the problem complexity in terms of the effort and time required to develop the product.

3.1 Lines of Code (LOC)

- LOC is possibly the simplest among all metrics available to measure project size.
- Consequently, this metric is extremely popular.
- This metric measures the size of a project by counting the number of source instructions in the developed program.
- Obviously, while counting the number of source instructions, comment lines, and header lines are ignored.
- Determining the LOC count at the end of a project is very simple.
- However, accurate estimation of LOC count at the beginning of a project is a very difficult task.
- One can possibly estimate the LOC count at the starting of a project, only by using some form of systematic guess work.
- Systematic guessing typically involves the following.

The project manager divides the problem into modules, and each module into sub-modules and so on, until the LOC of the leaf-level modules are small enough to be predicted. To be able to predict the LOC count for the various leaf-level modules sufficiently accurately, past experience in developing similar modules is very helpful.

Important shortcomings of the LOC metric in the following subsections:

1. LOC is a measure of coding activity alone.
2. LOC count depends on the choice of specific instructions:
3. LOC measure correlates poorly with the quality and efficiency of the code:
4. LOC metric penalises use of higher-level programming languages and code reuse

5. LOC metric measures the lexical complexity of a program and does not address the more important issues of logical and structural complexities:
6. It is very difficult to accurately estimate LOC of the final program from problem specification

3.2.Function Point (FP) Metric

The conceptual idea behind the function point metric is the following.

1. The size of a software product is directly dependent on the number of different high-level functions or features it supports.
2. This assumption is reasonable,since each feature would take additional effort to implement.

Though each feature takes some effort to develop, different features may take very different amounts of efforts to develop.

Function point (FP) metric computation

The size of a software product (in units of function points or FPs) is computed using different characteristics of the product identified in its requirements specification.

It is computed using the following three steps:

Step 1: Compute the unadjusted function point (UFP) using a heuristic expression.

Step 2: Refine UFP to reflect the actual complexities of the different parameters used in UFP computation.

Step 3: Compute FP by further refining UFP to account for the specific characteristics of the project that can influence the entire development effort.

4.SCHEDULING

Scheduling the project tasks is an important project planning activity.

- Once a schedule has been worked out and the project gets underway, the project manager monitors the timely completion of the tasks and takes any corrective action that may be necessary whenever there is a chance of schedule slippage.

<u>In order to schedule the project activities, a software project manager needs to do the following:</u>

1. Identify all the major activities that need to be carried out to complete the project.
2. Break down each activity into tasks.
3. Determine the dependency among different tasks.
4. Establish the estimates for the time durations necessary to complete the tasks.
5. Represent the information in the form of an activity network.
6. Determine task starting and ending dates from the information represented in the activity network.
7. Determine the critical path. A critical path is a chain of tasks that determines the duration of the project.
8. Allocate resources to tasks.

- The first step in scheduling a software project involves identifying all the activities necessary to complete the project.
- A good knowledge of the intricacies of the project and the development process helps the managers to effectively identify the important activities of the project.
- Next, the activities are broken down into a logical set of smaller activities (subactivities).
- The smallest subactivities are called tasks which are assigned to different developers.
- A project manager breakdowns the tasks systematically by using the work breakdown structure technique discussed in Section 3.10.1.
- After the project manager has broken down the activities into tasks, he has to find the dependency among the tasks.

- Dependency among the different tasks determines the order in which the different tasks would be carried out
- If a task A requires the results of another task B, then task A must be scheduled after task B and A is said to be dependent on B.
- In general, the task dependencies define a partial ordering among tasks.
- That is, each task may precede a subset of other tasks, but some tasks might not have any precedence ordering defined among them (called concurrent task).
- The dependency among the activities are represented in the form of an activity network discussed in Section 3.10.2.
- Once the activity network representation has been worked out, resources are allocated to each activity.
- Resource allocation is typically done using a Gantt chart. After resource allocation is done, a project evaluation and review technique (PERT) chart representation is developed.
- The PERT chart representation is useful to a project manager to carry out project monitoring and control.

4.1.Work Breakdown Structure

Let us now discuss the work break down structure, activity network, Gantt and PERT charts. First, let us understand why it is necessary to break down project activities into tasks.

- Once project activities have been decomposed into a set of tasks using WBS, the time frame when each activity is to be performed is to be determined.
- The end of each important activity is called a milestone.
- The project manager tracks the progress of a project by monitoring the timely completion of the milestones.
- If he observes that some milestones start getting delayed, he carefully monitors and controls the progress of the tasks,so that the overall deadline can still be met.
- WBS provides a notation for representing the activities,sub-activities, and tasks needed to be carried out in order to solve a problem.
- Each of these is represented using a rectangle (see Figure 3.8).

The root of the tree is labelled by the project name. Each node of the tree is broken down into smaller activities that are made the children of the node. To decompose an activity to a sub-activity, a good knowledge of the activity can be useful. Figure 3.8 represents the WBS of a management information system (MIS) software.

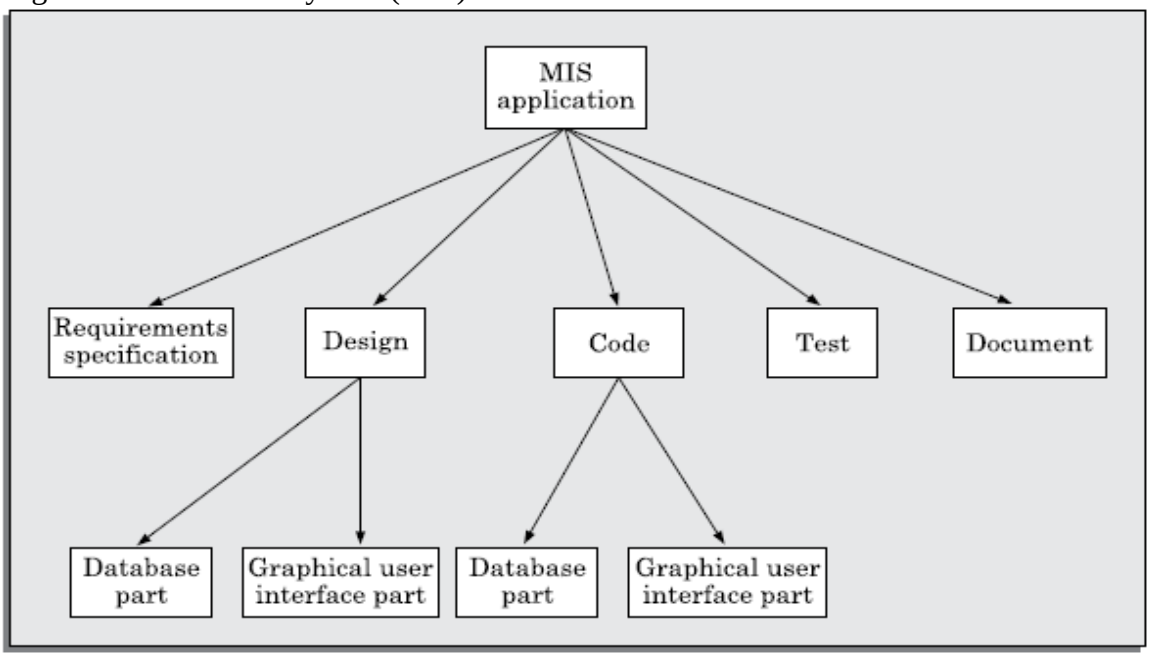


FIGURE 3.8 Work breakdown structure of an MIS problem.

How long to decompose?
The decomposition of the activities is carried out until any of the following is satisfied:

- „ A leaf-level subactivity (a task) requires approximately two weeks to develop.
- „ Hidden complexities are exposed, so that the job to be done is understood and can be assigned as a unit of work to one of the developers.
- „ Opportunities for reuse of existing software components is identified.

4.2 Activity Networks

1. An activity network shows the different activities making up a project, their estimated durations, and their interdependencies.
2. Two equivalent representations for activity networks are possible and are in use:
3. Activity on Node (AoN): In this representation, each activity is represented by a rectangular (some use circular) node and the duration of the activity is shown alongside each task in the node.
4. The inter-task dependencies are shown using directional edges (see Figure 3.9).
5. Activity on Edge (AoE): In this representation tasks are associated with the edges. The edges are also annotated with the task duration.
6. The nodes in the graph represent project milestones.
7. Activity networks were originally represented using activity on edge (AoE) representation.
8. However, later activity on node (AoN) has become popular since this representation is easier to understand and revise

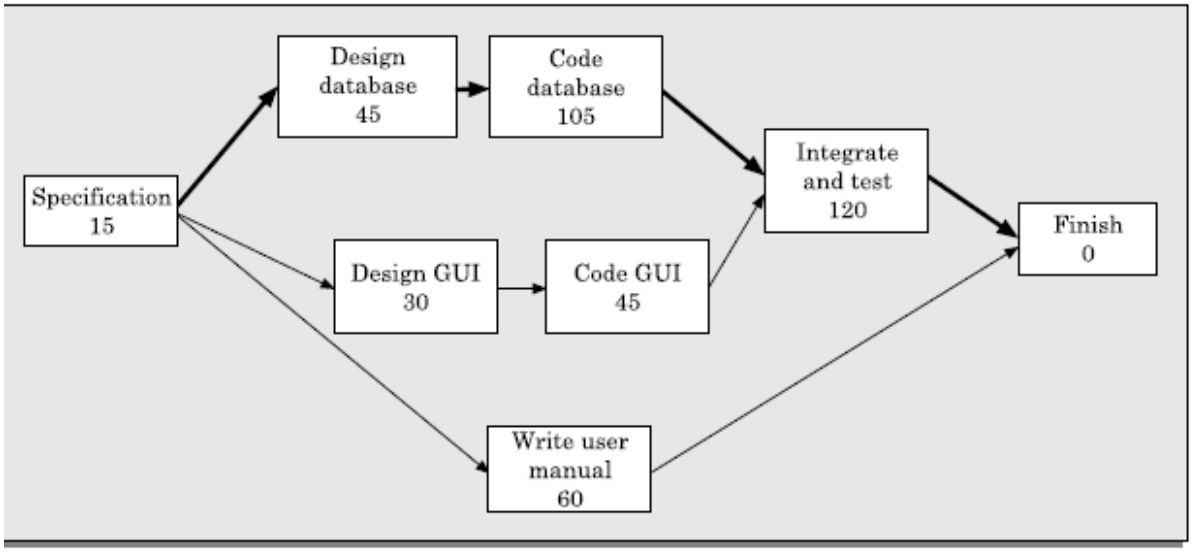


FIGURE 3.9 Activity network representation of the MIS problem.

4.3 **Critical Path Method (CPM)**

- CPM and PERT are operation research techniques that were developed in the late 1950s.
- Since then, they have remained extremely popular among project managers. Of late, these two techniques have got merged and many project management tools support them as CPM/PERT.
- However, we point out the fundamental differences between the two and discuss CPM in this subsection and PERT in the next subsection.
- A path in the activity network graph is any set of consecutive nodes and edges in this graph from the starting node to the last node.
- A critical path consists of a set of dependent tasks that need to be performed in a sequence and which together take the longest time to complete.

- CPM is an algorithmic approach to determine the critical paths and slack times for tasks not on the critical paths involves calculating the following quantities:
- Minimum time (MT): It is the minimum time required to complete the project.
- It is computed by determining the maximum of all paths from start to finish.
- Earliest start (ES): It is the time of a task is the maximum of all paths from the start to this task.
- The ES for a task is the ES of the previous task plus the duration of the preceding task.
- Latest start time (LST): It is the difference between MT and the maximum of all paths from this task to the finish.
- The LST can be computed by subtracting the duration of the subsequent task from the LST of the subsequent task.
- Earliest finish time (EF): The EF for a task is the sum of the earliest start time of the task and the duration of the task.
- Latest finish (LF): LF indicates the latest time by which a task can finish without affecting the final completion time of the project.
- A task completing beyond its LF would cause project delay. LF of a task can be obtained by subtracting maximum of all paths from this task to finish from MT.
- Slack time (ST): The slack time (or float time) is the total time that a task may be delayed before it will affect the end time of the project. The slack time indicates the ”flexibility” in starting and completion of tasks. ST for a task is LS-ES and can equivalently be written as LF-EF.

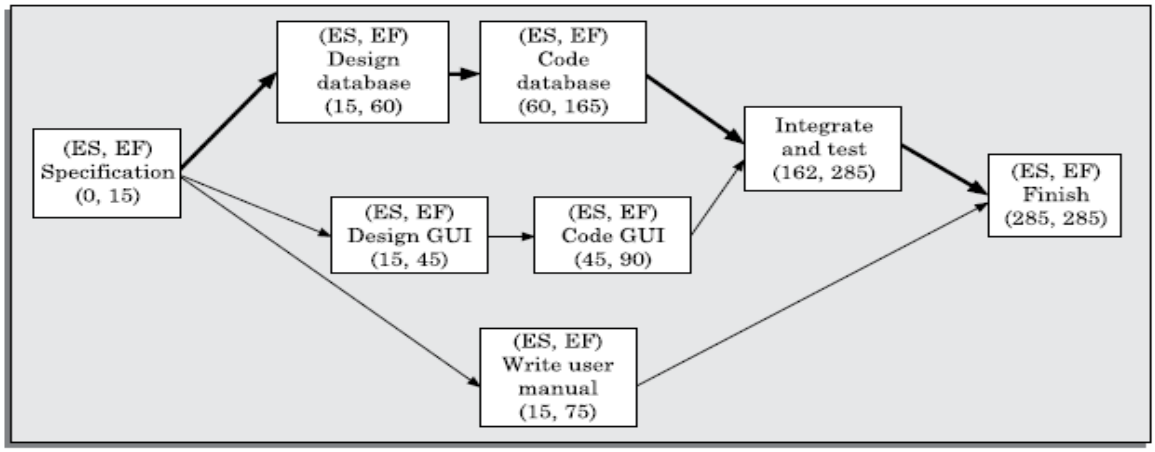


FIGURE 3.10 AoN for MIS problem with (ES, EF).

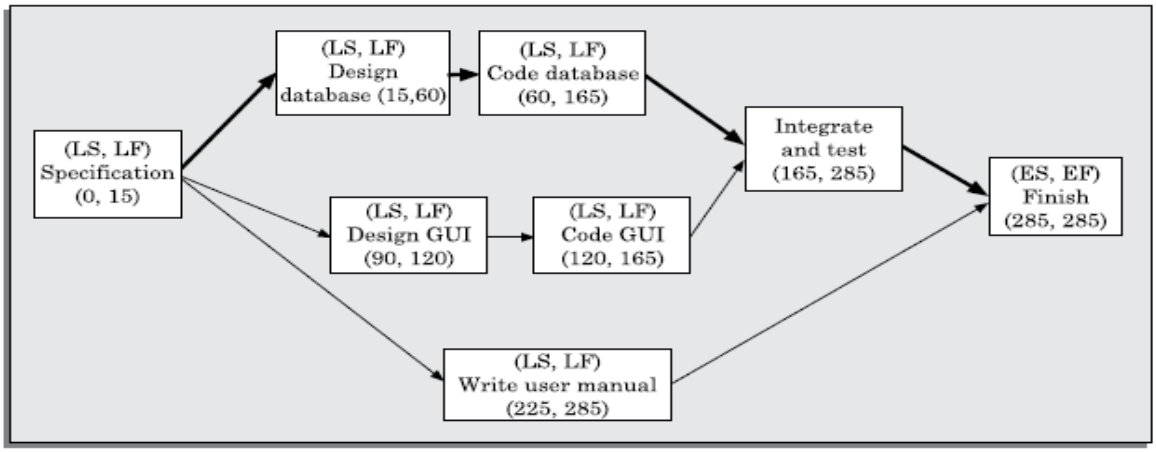


FIGURE 3.11 AoN of MIS problem with (LS, LF).

TABLE 3.8 Project Parameters Computed from Activity Network

Task	ES	EF	LS	LF	ST
Specification	0	15	0	15	0
Design data base	15	60	15	60	0
Design GUI part	15	45	90	120	75
Code data base	60	165	60	165	0
Code GUI part	45	90	120	165	75
Integrate and test	165	285	165	285	0
Write user manual	15	75	225	285	210

The critical paths are all the paths whose duration equals MT. The critical path in Figure 3.9 is shown using a thick arrows.

4.4PERT Charts

- Project evaluation and review technique (PERT) charts are a more sophisticated form of activity chart.
- Project managers know that there is considerable uncertainty about how much time a task would exactly take to complete.
- The duration assigned to tasks by the project manager are after all only estimates.
- Therefore, in reality the duration of an activity is a random variable with some probability distribution.
- In this context, PERT charts can be used to determine the probabilistic times for reaching various project mile stones, including the final mile stone.
- PERT charts like activity networks consist of a network of boxes and arrows.
- The boxes represent activities and the arrows represent task dependencies.
- A PERT chart represents the statistical variations in the project estimates assuming these to be normal distribution.
- PERT allows for some randomness in task completion times, and therefore provides the capability to determine the probability for achieving project milestones based on the probability of completing each task along the path to that milestone.
- Each task is annotated with three estimates:

„ Optimistic (O): The best possible case task completion time.
„ Most likely estimate (M): Most likely task completion time.
„ Worst case (W): The worst possible case task completion time.

The PERT chart representation of the MIS problem of Figure 3.9 is shown in Figure 3.12.

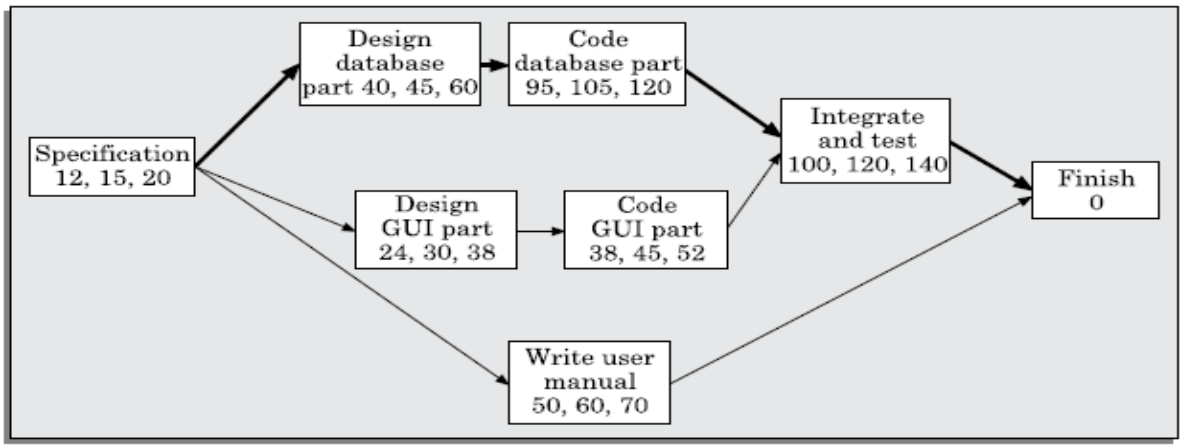


FIGURE 3.12 PERT chart representation of the MIS problem.

4.5. GANTT CHARTS

- Gantt chart has been named after its developer Henry Gantt. A Gantt chart is a form of bar chart.
- The vertical axis lists all the tasks to be performed. The bars are drawn along the y-axis, one for each task. Gantt charts used in software project management are actually an enhanced version of the standard Gantt charts. In the Gantt charts used for software project management, each bar consists of an unshaded part and a shaded part.

- The shaded part of the bar shows the length of time each task is estimated to take.
- The unshaded part shows the slack time or lax time.
- The lax time represents the leeway or flexibility available in meeting the latest time by which a task must be finished.
- A Gantt chart representation for the MIS problem of Figure 3.9 is shown in Figure 3.13.
- Gantt charts are useful for resource planning (i.e. allocate resources to activities).
- The different types of resources that need to be allocated to activities include staff, hardware, and software.

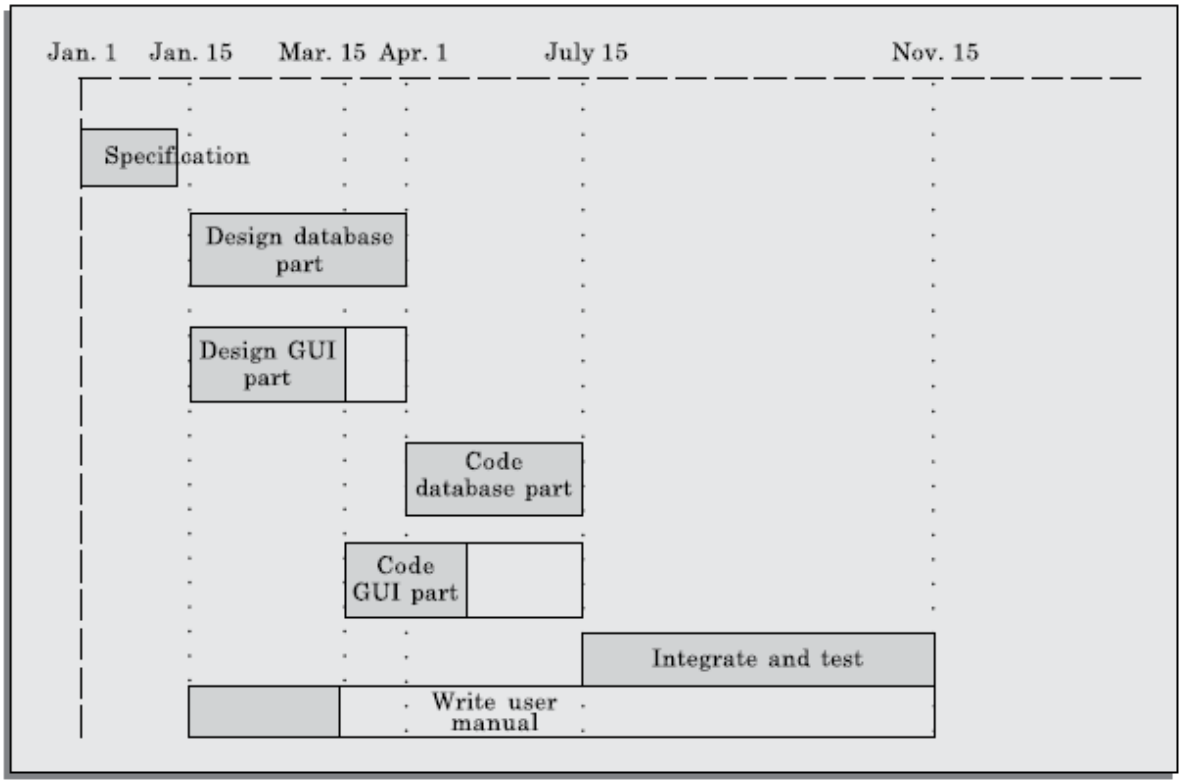


FIGURE 3.13 Gantt chart representation of the MIS problem.

5.SOFTWARE CONFIGURATION MANAGEMENT

The configuration of the software is the state of all project deliverables at any point of time; and software configuration management deals with effectively tracking and controlling the configuration of a software during its life cycle.

5.1 Necessity of Software Configuration Management

The following are some of the important problems that can crop up, if configuration management is not used:

There are several reasons for putting an object under configuration management.

Inconsistency problem when the objects are replicated:

- Consider a scenario where every software developer has a personal copy of an object (e.g. source code).

- When a developer makes changes to his local copy, he is expected to intimate the changes that he has made to other developers, so that the necessary changes in interfaces could be uniformly carried out across all modules.
- However, not only would it eat up valuable time of the developers,but many times a developer might make changes to the interfaces in his local copies and forgets to intimate the teammates about the changes.
- This makes the different copies of the object inconsistent. Finally, when the different modules are integrated, it does not work.
- Therefore, when several team members work on developing an application, it is necessary for them to work on a single copy of the application, otherwise inconsistencies may arise.

Problems associated with concurrent access:

- Possibly the most important reason for configuration management is to control the access to the different deliverable objects.
- Unless strict discipline is enforced regarding updation and storage of different objects, several problems can appear.
- Assume that only a single copy of a program module is maintained,and several developer are working on it.
- Two developers may simultaneously carry out changes to different functions of the same module, and while saving overwrite each other.

Similar problems can occur for any other deliverable object.

Providing a stable development environment:

- When a project work is underway, the team members need a stable environment to make progress.
- Suppose one developer is trying to integrate module A, with the modules B and C; since if developer of module C keeps changing C; this can be especially frustrating if a change to module C forces recompilation of the module.
- When an effective configuration management is in place, the manager freezes the objects to form a *baseline*.
- When any team member needs to change any of the objects under configuration control, he is provided with a copy of the baseline item.
- The requester makes changes to his private copy. Only after the requester is through with all modifications to his private copy, the configuration is updated and a new baseline gets formed instantly. This establishes a baseline for others to use and depend on.
- Also, baselines may be archived periodically (archiving means copying to a safe place such as a remote storage), so that the last baseline

can be recovered when there is a disaster.

System accounting and maintaining status information: System accounting denotes keeping track of who made a particular change to an object and when the change was made.

Handling variants: Existence of variants of a software product causes some peculiar problems. Suppose you have several variants of the same module, and find that a bug exists in one of them. Then, it has to be fixed in all versions and revisions. To do it efficiently, you should not have to fix it in each and every version and revision of the software separately. Making a change to one program should be reflected appropriately in all relevant versions and revisions.

5.2 Configuration Management Activities

Configuration management is carried out through two principal activities:

1. **Configuration identification:** It involves deciding which parts of the system should be kept track of.

2. **Configuration control:** It ensures that changes to a system happen smoothly.Normally, a project manager performs the configuration management activity by using a configuration management tool. In addition, a configuration management tool helps to keep track of various deliverable objects, so that the project manager can quickly and unambiguously determine the current state of the project. The configuration management tool enables the developer to change various components in a controlled manner.

In the following subsections, we provide an overview of the two configuration management activities.

1. Configuration identification

Project managers normally classify the objects associated with a software development into three main categories—controlled, precontrolled, and uncontrolled.

- Controlled objects are those that are already under configuration control. The team members must follow some formal procedures to change them.
- Precontrolled objects are not yet under configuration control, but will eventually be under configuration control.
- Uncontrolled objects are not subject to configuration control. Controllable objects include both controlled and precontrolled objects.

Typical controllable objects include:

1. Requirements specification document
2. Design documents
3. Tools used to build the system, such as compilers, linkers, lexical analysers, parsers,etc.
4. Source code for each module
5. Test cases
6. Problem reports

Configuration management plan is written during the project planning phase. It lists all controlled objects. The managers who develop the plan must strike a balance between controlling too much, and controlling too little. If too much is controlled, overheads due to configuration management increase to unreasonably high levels. On the other hand, controlling too little might lead to confusion and inconsistency when something changes.

2.Configuration control

- Configuration control is the process of managing changes to controlled objects.
- The configuration control part of a configuration management system that most directly affects the day-to-day operations of developers.
- In order to change a controlled object such as a module, a developer can get a private copy of the module by a *reserve* operation (see Figure 3.19).
- Configuration management tools allow only one person to reserve a module at any time.
- Once an object is reserved, it does not allow anyone else to reserve this module until the reserved module is *restored*.
- Thus, by preventing more than one developer to simultaneously reserve a module, the problems associated with concurrent access are solved.

Let us see how an object under configuration control can be changed. The developer needing to change a module first makes a *reserve* request.

After the reserve command successfully executes, a private copy of the module is created in his local directory.

- Then, he carries out all necessary changes on his private copy. Once he has satisfactorily made all the necessary changes, the changes need to be restored in configuration management repository.
- However, restoring the changed module to the system configuration requires the permission of a change control board (CCB). The CCB is usually constituted from among

the development team members.
For every change that needs to be carried out, the CCB reviews the changes made to the controlled object and certifies several things about the
change:
1. Change is well-motivated.
2. Developer has considered and documented the effects of the change.
3. Changes interact well with the changes made by other developers.
4. Appropriate people (CCB) have validated the change, i.e., someone has tested the changed code, and has verified that the change is consistent with the requirement.

- The change control board (CCB) sounds like a group of people. However, except for very large projects, the functions of the change control board are normally discharged by the project manager himself or some senior member of the development team. Once the CCB reviews the changes to the module, the project manager updates the old baseline through a restore operation (see Figure 3.19).
- A configuration control tool does not allow a developer to replace an object he has reserved with his local copy unless he gets an authorization from the CCB.
- By constraining the developers' ability to replace reserved objects, a stable environment is achieved. Since a configuration management tool allows only one developer to work on one module at any one time, problem of accidental overwriting is eliminated. Also, since only the manager can update the baseline after the CCB approval, unintentional changes to the configuration items are eliminated.

Software Project Management

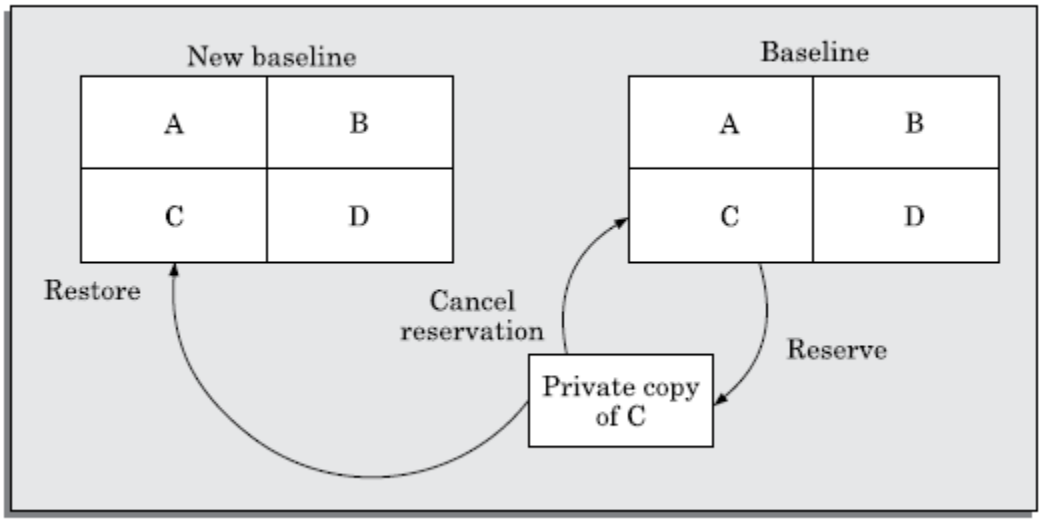


FIGURE 3.19 Reserve and restore operation in configuration control.

Source code control system (SCCS) and RCS Revision **control system**
SCCS and RCS are two popular configuration management tools available on most UNIX systems.

- SCCS or RCS can be used for controlling and managing different versions of text files. SCCS and RCS do not handle binary files (i.e., executable files, documents, files containing diagrams, etc.)
- SCCS and RCS provide an efficient way of storing versions that minimises the amount of occupied disk space. Suppose, a module MOD is present in 3 versions—MOD1.1, MOD1.2 and MOD1.3. Then, SCCS and RCS stores the original module MOD1.1 together with changes needed to transform MOD1.1 into MOD1.2 and MOD1.2 to MOD1.3. The changes needed to transform each baseline file to the next version are stored and are called deltas. The main reason behind storing the deltas rather than storing the

full revision files is to save disk space.

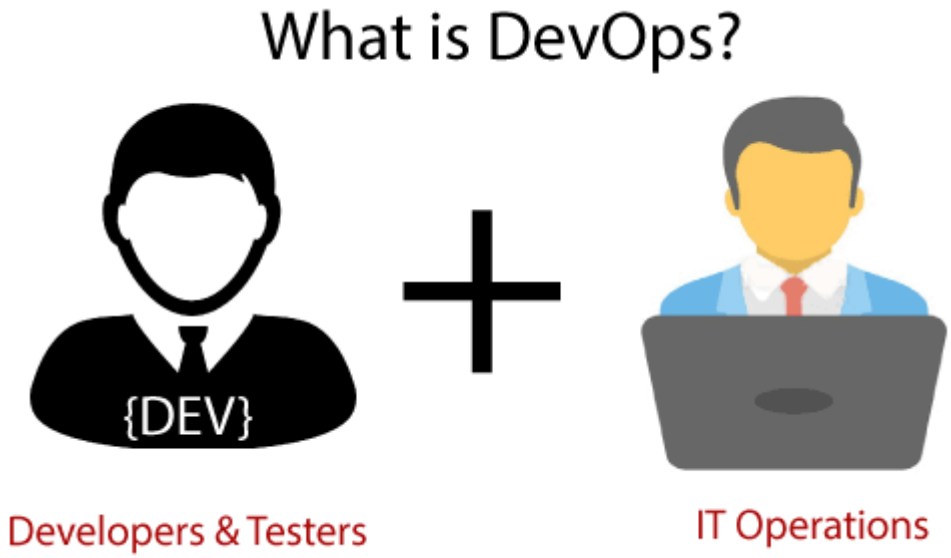
- The change control facilities provided by SCCS and RCS include the ability to incorporate restrictions on the set of individuals who can create new versions, and facilities for checking components in and out through reserve and restore operations. Individual developers check out components and modify them.
- After they have made all necessary changes to a module and after the changes have been reviewed, they check in the changed module into SCCS or RCS. Revisions of a configuration item are denoted by numbers in ascending order, e.g., 1.1, 1.2, 1.3, etc. It is also possible to create variants or revisions of a component by creating a fork in the development history.

6.DevOps

The DevOps is the combination of two words, one is **Development** and other is **Operations**. It is a culture to promote the development and operation process collectively.

What is DevOps?

The DevOps is a combination of two words, one is software Development, and second is Operations. This allows a single team to handle the entire application lifecycle, from development to **testing, deployment**, and **operations**. DevOps helps you to reduce the disconnection between software developers, quality assurance (QA) engineers, and system administrators.



1. DevOps promotes collaboration between Development and Operations team to deploy code to production faster in an automated & repeatable way.
2. DevOps can also be defined as a sequence of development and IT operations with better communication and collaboration.
3. DevOps has become one of the most valuable business disciplines for enterprises or organizations. With the help of DevOps, **quality**, and **speed** of the application delivery has improved to a great extent.
4. DevOps is nothing but a practice or methodology of making "**Developers**" and "**Operations**" folks work together. DevOps represents a change in the IT culture with

a complete focus on rapid IT service delivery through the adoption of agile practices in the context of a system-oriented approach.

5. DevOps is all about the integration of the operations and development process. Organizations that have adopted DevOps noticed a 22% improvement in software quality and a 17% improvement in application deployment frequency and achieve a 22% hike in customer satisfaction. 19% of revenue hikes as a result of the successful DevOps implementation.

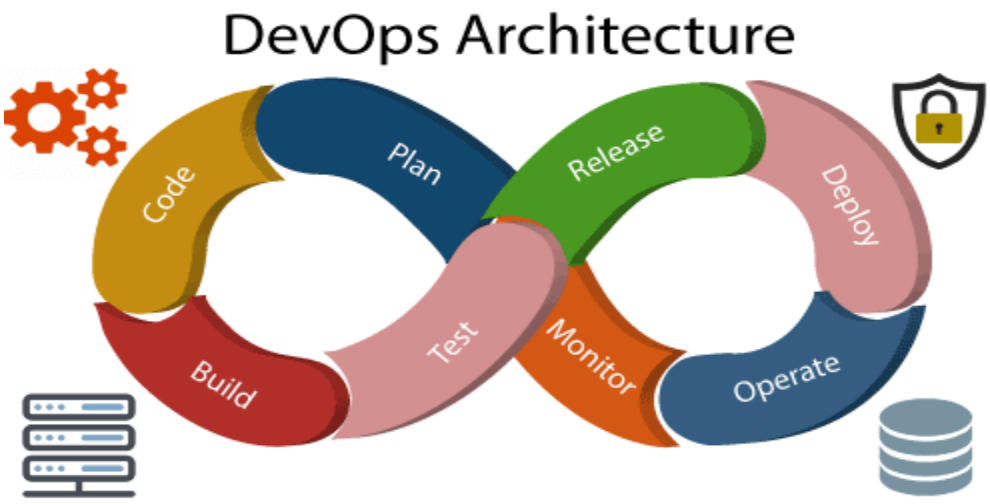
Why DevOps?

1. The operation and development team worked in complete isolation.
2. After the design-build, the testing and deployment are performed respectively. That's why they consumed more time than actual build cycles.
3. Without the use of DevOps, the team members are spending a large amount of time on designing, testing, and deploying instead of building the project.
4. Manual code deployment leads to human errors in production.
5. Coding and operation teams have their separate timelines and are not in synch, causing further delays.

DevOps History

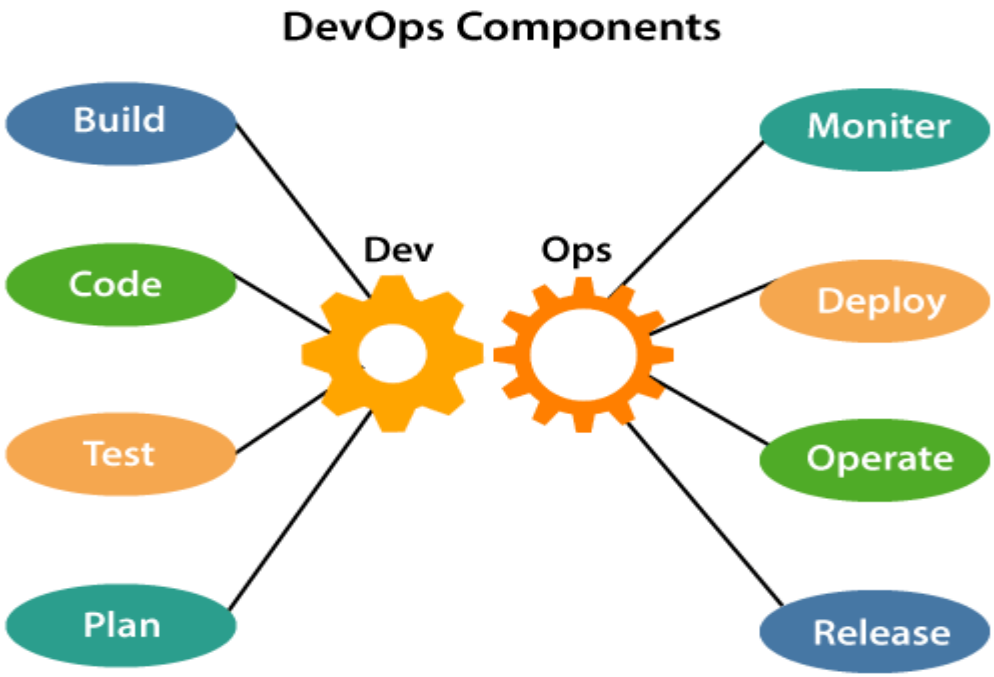
- In 2009, the first conference named **DevOpsdays** was held in Ghent Belgium. Belgian consultant and Patrick Debois founded the conference.
- In 2012, the state of DevOps report was launched and conceived by Alanna Brown at Puppet.
- In 2014, the annual State of DevOps report was published by Nicole Forsgren, Jez Humble, Gene Kim, and others. They found DevOps adoption was accelerating in 2014 also.
- In 2015, Nicole Forsgren, Gene Kim, and Jez Humble founded DORA (DevOps Research and Assignment).
- In 2017, Nicole Forsgren, Gene Kim, and Jez Humble published "Accelerate: Building and Scaling High Performing Technology Organizations".

DevOps Architecture



1. Development and operations both play essential roles in order to deliver applications. The deployment comprises analyzing the **requirements, designing, developing**, and **testing** of the software components or frameworks.
2. The operation consists of the administrative processes, services, and support for the software. When both the development and operations are combined with collaborating, then the DevOps architecture is the solution to fix the gap between deployment and operation terms; therefore, delivery can be faster.
3. DevOps architecture is used for the applications hosted on the cloud platform and large distributed applications. Agile Development is used in the DevOps architecture so that integration and delivery can be contiguous. When the development and operations team works separately from each other, then it is time-consuming to **design, test**, and **deploy**. And if the terms are not in sync with each other, then it may cause a delay in the delivery. So DevOps enables the teams to change their shortcomings and increases productivity.

Below are the various components that are used in the DevOps architecture:



1) Build

Without DevOps, the cost of the consumption of the resources was evaluated based on the pre-defined individual usage with fixed hardware allocation. And with DevOps, the usage of cloud, sharing of resources comes into the picture, and the build is dependent upon the user's need, which is a mechanism to control the usage of resources or capacity.

2) Code

Many good practices such as Git enables the code to be used, which ensures writing the code for business, helps to track changes, getting notified about the reason behind the difference in the actual and the expected output, and if necessary reverting to the original code developed. The code can be appropriately arranged in **files, folders**, etc. And they can be reused.

3) Test

The application will be ready for production after testing. In the case of manual testing, it consumes more time in testing and moving the code to the output. The testing can be automated, which decreases the time for testing so that the time to deploy the code to production can be reduced as automating the running of the scripts will remove many manual steps.

4) Plan

DevOps use Agile methodology to plan the development. With the operations and development team in sync, it helps in organizing the work to plan accordingly to increase productivity.

5) Monitor

Continuous monitoring is used to identify any risk of failure. Also, it helps in tracking the system accurately so that the health of the application can be checked. The monitoring becomes more comfortable with services where the log data may get monitored through many third-party tools such as **Splunk**.

6) Deploy

Many systems can support the scheduler for automated deployment. The cloud management platform enables users to capture accurate insights and view the optimization scenario, analytics on trends by the deployment of dashboards.

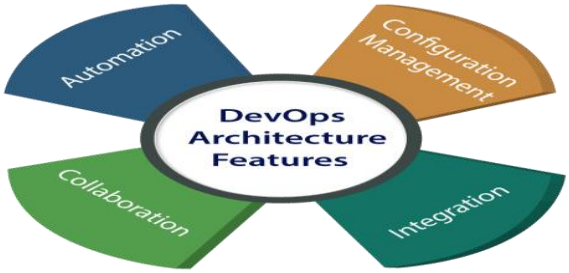
7) Operate

DevOps changes the way traditional approach of developing and testing separately. The teams operate in a collaborative way where both the teams actively participate throughout the service lifecycle. The operation team interacts with developers, and they come up with a monitoring plan which serves the IT and business requirements.

8) Release

Deployment to an environment can be done by automation. But when the deployment is made to the production environment, it is done by manual triggering. Many processes involved in release management commonly used to do the deployment in the production environment manually to lessen the impact on the customers.

DevOps Architecture Features
Here are some key features of DevOps architecture, such as:



1) Automation

Automation can reduce time consumption, especially during the testing and deployment phase. The productivity increases, and releases are made quicker by automation. This will lead in catching bugs quickly so that it can be fixed easily. For contiguous delivery, each code is defined through automated tests, cloud-based services, and builds. This promotes production using automated deploys.

2) Collaboration

The Development and Operations team collaborates as a DevOps team, which improves the cultural model as the teams become more productive with their productivity, which strengthens accountability and ownership. The teams share their responsibilities and work closely in sync, which in turn makes the deployment to production faster.

3) Integration

Applications need to be integrated with other components in the environment. The integration phase is where the existing code is combined with new functionality and then tested. Continuous integration and testing enable continuous development. The frequency in the releases and micro-services leads to significant operational challenges. To overcome such problems, continuous integration and delivery are implemented to deliver in a **quicker, safer**, and **reliable manner**.

4) Configuration management

It ensures the application to interact with only those resources that are concerned with the environment in which it runs. The configuration files are not created where the external configuration to the application is separated from the source code. The configuration file can be written during deployment, or they can be loaded at the run time, depending on the environment in which it is running.

DevOps Advantages and Disadvantages

Here are some advantages and disadvantages that DevOps can have for business, such as:

Advantages

- DevOps is an excellent approach for quick development and deployment of applications.
- It responds faster to the market changes to improve business growth.
- DevOps escalate business profit by decreasing software delivery time and transportation costs.
- DevOps clears the descriptive process, which gives clarity on product development and delivery.
- It improves customer experience and satisfaction.
- DevOps simplifies collaboration and places all tools in the cloud for customers to access.

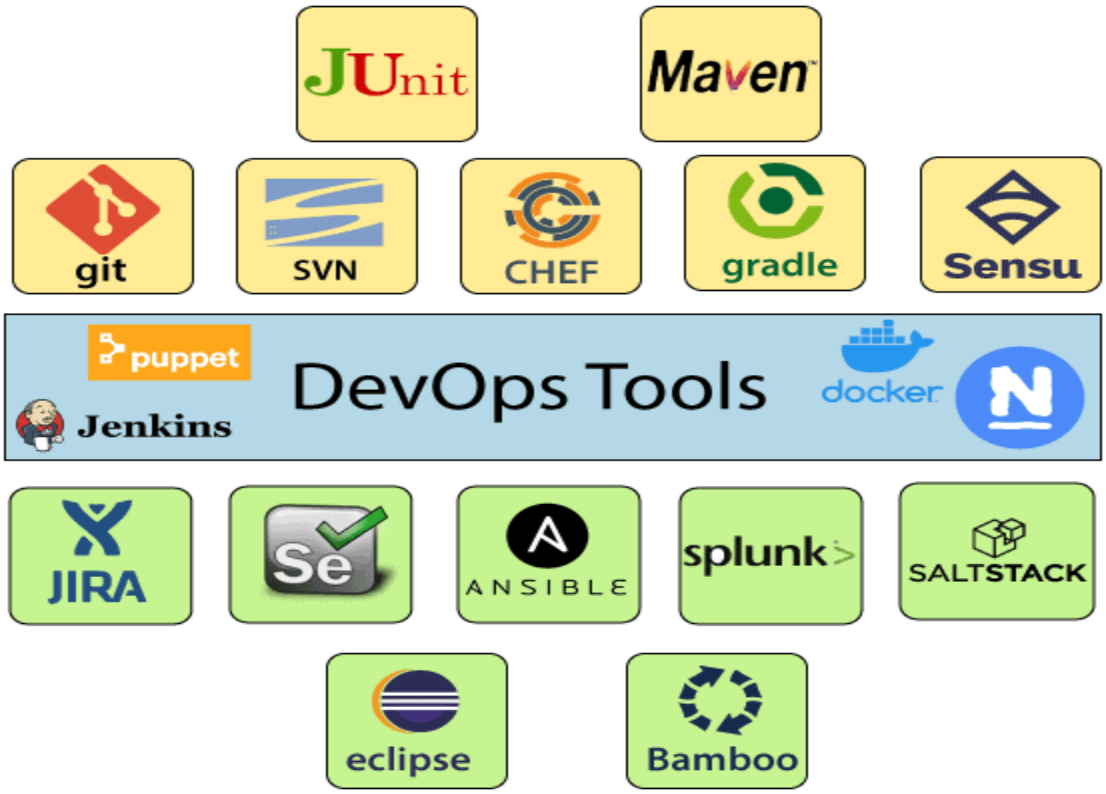
- DevOps means collective responsibility, which leads to better team engagement and productivity.

Disadvantages

- DevOps professional or expert's developers are less available.
- Developing with DevOps is so expensive.
- Adopting new DevOps technology into the industries is hard to manage in short time.
- Lack of DevOps knowledge can be a problem in the continuous integration of automation projects.

DevOps Tools

Here are some most popular DevOps tools with brief explanation shown in the below image, such as:



1) Puppet

Puppet is the most widely used DevOps tool. It allows the delivery and release of the technology changes quickly and frequently. It has features of versioning, automated testing, and continuous delivery. It enables to manage entire infrastructure as code without expanding the size of the team.

Features

- Real-time context-aware reporting.
- Model and manage the entire environment.
- Defined and continually enforce infrastructure.

- Desired state conflict detection and remediation.
- It inspects and reports on packages running across the infrastructure.
- It eliminates manual work for the software delivery process.
- It helps the developer to deliver great software quickly.

2) Ansible

Ansible is a leading DevOps tool. Ansible is an open-source IT engine that automates application deployment, cloud provisioning, intra service orchestration, and other IT tools. It makes it easier for DevOps teams to scale automation and speed up productivity.

Ansible is easy to deploy because it does not use any **agents** or **custom security** infrastructure on the client-side, and by pushing modules to the clients. These modules are executed locally on the client-side, and the output is pushed back to the Ansible server.

Features

- It is easy to use to open source deploy applications.
- It helps in avoiding complexity in the software development process.
- It eliminates repetitive tasks.
- It manages complex deployments and speeds up the development process.

3) Docker

Docker is a high-end DevOps tool that allows building, ship, and run distributed applications on multiple systems. It also helps to assemble the apps quickly from the components, and it is typically suitable for container management.

Features

- It configures the system more comfortable and faster.
- It increases productivity.
- It provides containers that are used to run the application in an isolated environment.
- It routes the incoming request for published ports on available nodes to an active container. This feature enables the connection even if there is no task running on the node.
- It allows saving secrets into the swarm itself.

4) Nagios

Nagios is one of the more useful tools for DevOps. It can determine the errors and rectify them with the help of network, infrastructure, server, and log monitoring systems.

Features

- It provides complete monitoring of desktop and server operating systems.

- The network analyzer helps to identify bottlenecks and optimize bandwidth utilization.
- It helps to monitor components such as services, application, OS, and network protocol.
- It also provides to complete monitoring of Java Management Extensions.

5) CHEF

A chef is a useful tool for achieving scale, speed, and consistency. The chef is a cloud-based system and open source technology. This technology uses Ruby encoding to develop essential building blocks such as recipes and cookbooks. The chef is used in infrastructure automation and helps in reducing manual and repetitive tasks for infrastructure management.

Chef has got its convention for different building blocks, which are required to manage and automate infrastructure.

Features

- It maintains high availability.
- It can manage multiple cloud environments.
- It uses popular Ruby language to create a domain-specific language.
- The chef does not make any assumptions about the current status of the node. It uses its mechanism to get the current state of the machine.

6) Jenkins

Jenkins is a DevOps tool for monitoring the execution of repeated tasks. Jenkins is a software that allows continuous integration. Jenkins will be installed on a server where the central build will take place. It helps to integrate project changes more efficiently by finding the issues quickly.

Features

- Jenkins increases the scale of automation.
- It can easily set up and configure via a web interface.
- It can distribute the tasks across multiple machines, thereby increasing concurrency.
- It supports continuous integration and continuous delivery.
- It offers 400 plugins to support the building and testing any project virtually.
- It requires little maintenance and has a built-in GUI tool for easy updates.

7) Git

Git is an open-source distributed version control system that is freely available for everyone. It is designed to handle minor to major projects with speed and efficiency. It is developed to co-ordinate the work among programmers. The version control allows you to track and work together with your team members at the same workspace. It is used as a critical distributed version-control for the DevOps tool.

Features

- It is a free open source tool.
- It allows distributed development.
- It supports the pull request.
- It enables a faster release cycle.
- Git is very scalable.
- It is very secure and completes the tasks very fast.

8) SALTSTACK

Stackify is a lightweight DevOps tool. It shows real-time error queries, logs, and more directly into the workstation. SALTSTACK is an ideal solution for intelligent orchestration for the software-defined data center.

Features

- It eliminates messy configuration or data changes.
- It can trace detail of all the types of the web request.
- It allows us to find and fix the bugs before production.
- It provides secure access and configures image caches.
- It secures multi-tenancy with granular role-based access control.
- Flexible image management with a private registry to store and manage images.

9) Splunk

Splunk is a tool to make machine data usable, accessible, and valuable to everyone. It delivers operational intelligence to DevOps teams. It helps companies to be more secure, productive, and competitive.

T

Features

- It has the next-generation monitoring and analytics solution.
- It delivers a single, unified view of different IT services.
- Extend the Splunk platform with purpose-built solutions for security.
- Data drive analytics with actionable insight.

10) Selenium

Selenium is a portable software testing framework for web applications. It provides an easy interface for developing automated tests.

Features

- It is a free open source tool.

- It supports multiplatform for testing, such as Android and ios.
- It is easy to build a keyword-driven framework for a WebDriver.
- It creates robust browser-based regression automation suites and tests.

DevOps Automation

Automation is the crucial need for DevOps practices, and automate everything is the fundamental principle of DevOps. Automation kick starts from the code generation on the developers machine, until the code is pushed to the code and after that to monitor the application and system in the production.

Automating infrastructure set up and configurations, and software deployment is the key highlight of DevOps practice. DevOps practice id is dependent on automation to make deliveries over a few hours and make frequent deliveries across platforms.

Automation in DevOps boosts speed, consistency, higher accuracy, reliability, and increases the number of deliveries. Automation in DevOps encapsulates everything right from the building, deploying, and monitoring.

DevOps Automation Tools

In large DevOps team that maintain extensive massive IT infrastructure can be classified into six categories, such as:

- Infrastructure Automation
- Configuration Management
- Deployment Automation
- Performance Management
- Log management
- Monitoring

Below are few tools in each of these categories let see in brief, such as:

Infrastructure Automation

Amazon Web Services (AWS): Being a cloud service, you don't need to be physically present in the data center, they are easy to scale on-demand, and there are no up-front hardware costs. It can be configured to provide more servers based on traffic automatically.

Configuration Management

Chef: Chef is a handy DevOps tool for achieving speed, scale, and consistency. It can be used to ease out of complex tasks and perform configuration management. With the help of this tool, the DevOps team can avoid making changes across ten thousand servers. Rather, they need to make changes in one place, which is automatically reflected in other servers.

Deployment Automation

Jenkins: It facilitates continuous integration and testing. It helps to integrate project changes more efficiently by quickly finding issues as soon as built is deployed.

Performance Management

App Dynamic: It offers real-time performance monitoring. The data collected by this tool help developers to debug when issues occur.

Log Management

Splunk: This DevOps tool solves issues such as storing, aggregating, and analyzing all logs in one place.

Monitoring

Nagios: It notified people when infrastructure and related service go down. Nagios is a tool for this purpose, which helps the DevOps team to find and correct problems.

DevOps Pipeline

A pipeline in software engineering team is a set of automated processes which allows DevOps professionals and developer to reliably and efficiently compile, build, and deploy their code to their production compute platforms.

The most common components of a pipeline in DevOps are build automation or continuous integration, test automation, and deployment automation.

A pipeline consists of a set of tools which are classified into the following categories such as:

- Source control
- Build tools
- Containerization
- Configuration management
- Monitoring

Continuous Integration Pipeline

Continuous integration (CI) is a practice in which developers can check their code into a version-controlled repository several times per day. Automated build pipelines are triggered by these checks which allows fast and easy to locate error detection.

Some significant benefits of CI are:

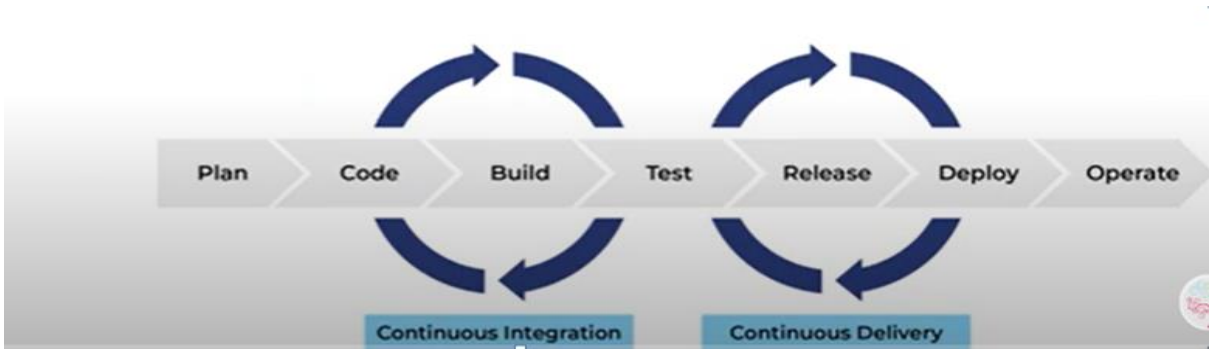
- Small changes are easy to integrate into large codebases.
- More comfortable for other team members to see what you have been working.
- Fewer integration issues allowing rapid code delivery.
- Bugs are identified early, making them easier to fix, resulting in less debugging work.

Continuous Delivery Pipeline

Continuous delivery (CD) is the process that allows operation engineers and developers to deliver bug fixes, features, and configuration change into production reliably, quickly, and sustainably. Continuous delivery offers the benefits of code delivery pipelines, which are carried out that can be performed on demand.

Some significant benefits of the CD are:

- Faster bug fixes and features delivery.
- CD allows the team to work on features and bug fixes in small batches, which means user feedback received much quicker. It reduces the overall time and cost of the project.



DevOps Methodology

We have a demonstrated methodology that takes an approach to cloud adoption. It accounts for all the factors required for successful approval such as people, process, and technology, resulting in a focus on the following critical consideration:

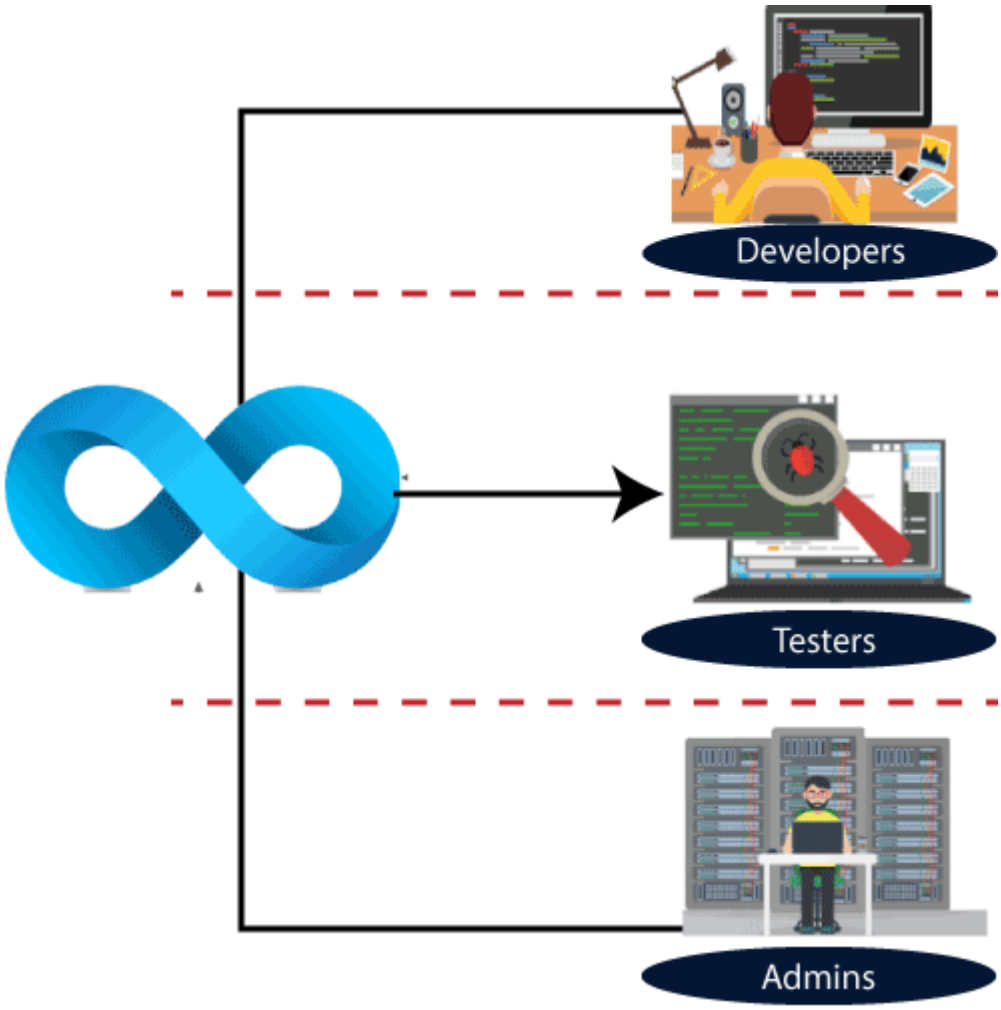
- **The Teams:** Mission or project and cloud management.
- **Connectivity:** Public, on-premise, and hybrid cloud network access.
- **Automation:** Infrastructure as code, scripting the orchestration and deployment of resources.
- **On-boarding Process:** How the project gets started in the cloud.
- **Project Environment:** TEST, DEV, PROD (identical deployment, testing, and production).
- **Shared Services:** Common capabilities provided by the enterprise.
- **Naming Conventions:** Vital aspect to track resource utilization and billing.
- **Defining Standards Role across the Teams:** Permissions to access resources by job function.

DevOps Engineers

DevOps Engineer is an IT professional who works with system operators, software developers, and other production IT staff to administer code releases.

DevOps engineer understands the software development lifecycle and various automation tools for developing digital pipelines.

DevOps have hard as well as soft skills to communicate and collaborate with development, testing, and operations teams.



DevOps engineers need to code occasionally from scratch, and they must have the basics of software development languages.

The DevOps engineer will work with development team staff to tackle the coding and scripting needed to connect elements of code, like libraries or software development kits.

A bachelor's degree in computer science or related fields is generally required for DevOps engineers. Many companies prefer those who have a master's degree and at least three to five years of work experience in this field. HTTP, HTML, CSS, SSL, XML, Linux, Java, Amazon Web Services (AWS), NoSQL technologies, DNS, and web app development.

DevOps Engineer Roles and Responsibilities

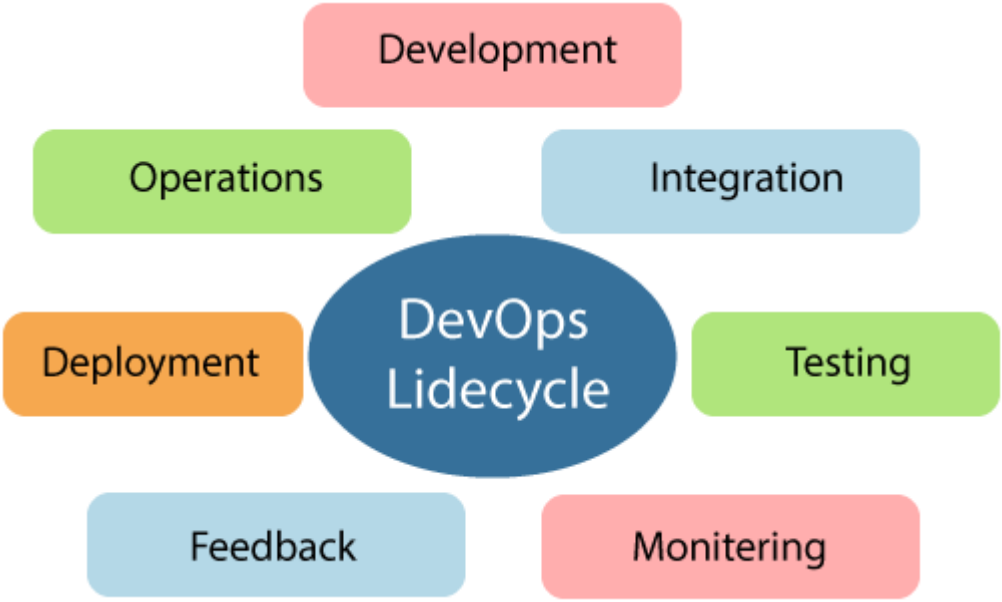
DevOps engineers work full time. They are responsible for the production and continuing maintenance of a software application platform.

Below are some roles, responsibilities, and skills which are expected from DevOps engineers, such as:

- Manage projects effectively through an open standard based platform.
- Increases project visibility through traceability.
- Improve quality and reduce the development cost with collaboration.
- DevOps should have the soft skill of problem solver and a quick learner.
- Analyze, design, and evaluate automation scripts and systems.
- Able to perform system troubleshooting and problem-solving across the platform and application domains.
- Ensuring the critical resolution of system issues by using the best cloud security solution services.

DevOps Lifecycle(OPEARTIONS)

DevOps defines an agile relationship between operations and Development. It is a process that is practiced by the development team and operational engineers together from beginning to the final stage of the product.



Learning DevOps is not complete without understanding the DevOps lifecycle phases. The DevOps lifecycle includes seven phases as given below:

1) Continuous Development

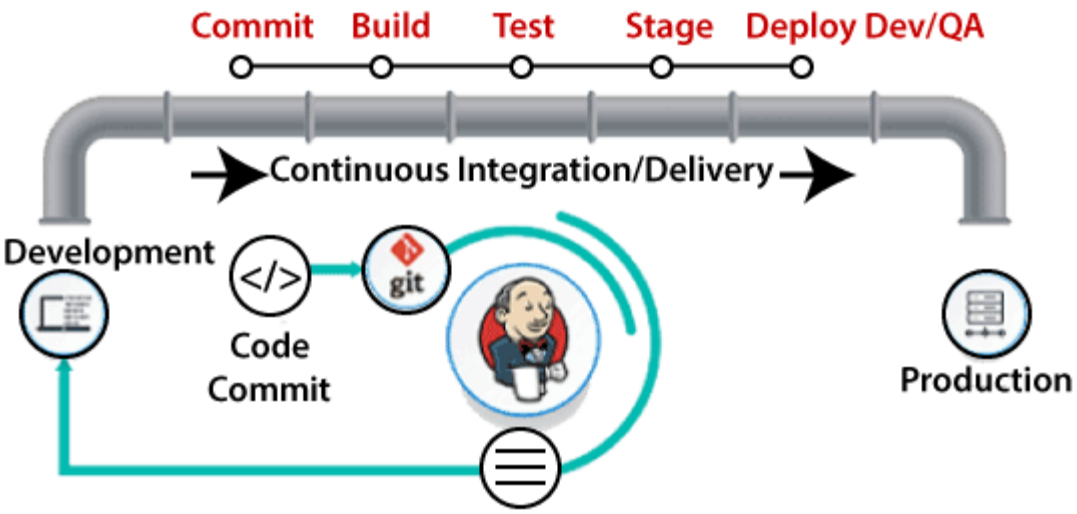
This phase involves the planning and coding of the software. The vision of the project is decided during the planning phase. And the developers begin developing the code for the application. There are no DevOps tools that are required for planning, but there are several tools for maintaining the code.

2) Continuous Integration

This stage is the heart of the entire DevOps lifecycle. It is a software development practice in which the developers require to commit changes to the source code more frequently. This may be on a daily or weekly basis. Then every commit is built, and this allows early detection

of problems if they are present. Building code is not only involved compilation, but it also includes **unit testing, integration testing, code review**, and **packaging**.

The code supporting new functionality is continuously integrated with the existing code. Therefore, there is continuous development of software. The updated code needs to be integrated continuously and smoothly with the systems to reflect changes to the end-users.



Jenkins is a popular tool used in this phase. Whenever there is a change in the Git repository, then Jenkins fetches the updated code and prepares a build of that code, which is an executable file in the form of war or jar. Then this build is forwarded to the test server or the production server.

3) Continuous Testing

This phase, where the developed software is continuously testing for bugs. For constant testing, automation testing tools such as **TestNG, JUnit, Selenium**, etc are used. These tools allow QAs to test multiple code-bases thoroughly in parallel to ensure that there is no flaw in the functionality. In this phase, **Docker** Containers can be used for simulating the test environment.

Selenium does the automation testing, and TestNG generates the reports. This entire testing phase can automate with the help of a Continuous Integration tool called **Jenkins**.

Automation testing saves a lot of time and effort for executing the tests instead of doing this manually. Apart from that, report generation is a big plus. The task of evaluating the test cases that failed in a test suite gets simpler. Also, we can schedule the execution of the test cases at predefined times. After testing, the code is continuously integrated with the existing code.

4) Continuous Monitoring

Monitoring is a phase that involves all the operational factors of the entire DevOps process, where important information about the use of the software is recorded and carefully processed to find out trends and identify problem areas. Usually, the monitoring is integrated within the operational capabilities of the software application.

It may occur in the form of documentation files or maybe produce large-scale data about the application parameters when it is in a continuous use position. The system errors such as server not reachable, low memory, etc are resolved in this phase. It maintains the security and availability of the service.

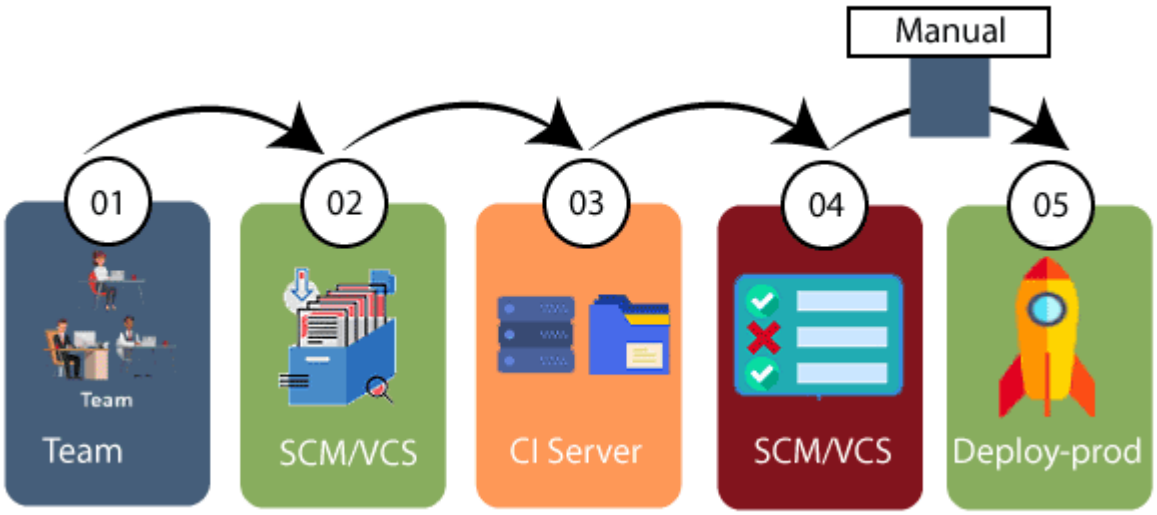
5) Continuous Feedback

The application development is consistently improved by analyzing the results from the operations of the software. This is carried out by placing the critical phase of constant feedback between the operations and the development of the next version of the current software application.

The continuity is the essential factor in the DevOps as it removes the unnecessary steps which are required to take a software application from development, using it to find out its issues and then producing a better version. It kills the efficiency that may be possible with the app and reduce the number of interested customers.

6) Continuous Deployment

In this phase, the code is deployed to the production servers. Also, it is essential to ensure that the code is correctly used on all the servers.



The new code is deployed continuously, and configuration management tools play an essential role in executing tasks frequently and quickly. Here are some popular tools which are used in this phase, such as **Chef, Puppet, Ansible**, and **SaltStack**.

Containerization tools are also playing an essential role in the deployment phase. **Vagrant** and **Docker** are popular tools that are used for this purpose. These tools help to produce consistency across development, staging, testing, and production environment. They also help in scaling up and scaling down instances softly.

Containerization tools help to maintain consistency across the environments where the application is tested, developed, and deployed. There is no chance of errors or failure in the production environment as they package and replicate the same dependencies and packages used in the testing, development, and staging environment. It makes the application easy to run on different computers.

7) Continuous Operations

All DevOps operations are based on the continuity with complete automation of the release process and allow the organization to accelerate the overall time to market continuingly.

It is clear from the discussion that continuity is the critical factor in the DevOps in removing steps that often distract the development, take it longer to detect issues and produce a better version of the product after several months. With DevOps, we can make any software product more efficient and increase the overall count of interested customers in your product.